

CSE 165 Project 3: Design Document

Project summary

This project implements a gesture aware embodied agent for Meta Quest 2 with video passthrough enabled. The user scans the real room, aims the right hand toward a reachable point on the floor, confirms the command with a left hand pinch, and watches an animated Mixamo character walk to that point through the captured room. The agent treats the anchored floor and walls as solid surfaces, so it stops at a wall instead of passing through it.

Submission links

GitHub: <https://github.com/halvis82/cse165-project3>

Video: <https://www.youtube.com/watch?v=UGWLjcqfzd8>

AI use disclosure

We used Claude (Anthropic) as an AI assistant during this project in the following ways:

- Scene bootstrapping: AI assisted in writing the Unity Editor and runtime bootstrap scripts that configure the AR Session, XR Origin, AR camera, passthrough-ready camera background, raycast manager, plane manager, anchor manager, scene hierarchy, materials, and build settings.
- Spatial anchors and physical space modeling: AI assisted in writing the room surface capture code that detects horizontal and vertical planes, attaches anchors, creates visible floor and wall proxies, places anchor markers, requests Quest scene permissions, and gives the wall and floor proxies solid colliders.
- Hand tracking and gesture commands: AI assisted in writing the hand joint utility code, hand joint visualizer, right-hand aim ray logic, left thumb-index pinch recognition, destination resolution, reticle preview, aim line, and in-headset status panel.
- Embodied agent behavior: AI assisted in writing the Mixamo character import support, material and texture rebinding, double-sided unlit shader, animation driver, movement controller, wall-blocking collision checks, and editor fallback controls used for iteration.
- Design document: AI assisted in drafting and typesetting this document, including the storyboard diagrams, heuristic evaluation table, implementation coverage, and submission links.

All generated code was reviewed, tested in Unity and on the Meta Quest 2, and debugged by the team. We understand the implementation and can explain any part of it.

Implemented system

- Passthrough mixed reality scene. The headset camera feed is visible behind the virtual content.
- Real room surfaces captured as spatial anchors. The floor and walls come from the headset Space Setup and are detected through AR Foundation plane tracking on the Meta OpenXR runtime.
- Anchored floor and wall game objects that act as digital copies of the physical surfaces. Each one carries a solid collider so the agent collides with it.
- Joint based gesture recognition. The right hand supplies an aim ray from its tracked joints, and a left thumb and index pinch confirms the destination.
- Mixamo Beetlejuice character with body and head textures reapplied at runtime. The agent turns, walks toward the target, arrives, and stops when a wall blocks the route.
- In headset status panel that reports captured floor and wall counts, the gesture state, and the navigation state.

Design exploration

The assignment asks for at least three storyboards that show how a hand gesture can indicate where the embodied agent should travel. The three techniques below were considered. The first one is the technique that was implemented.

Technique 1, implemented: aim the right hand, then pinch the left to confirm

The user aims the right hand toward a point on the floor. The aim ray follows the direction of the hand, so any hand pose works, whether the fingers are open, pointing, or closed. A reticle previews the floor point that the ray hits. The

user then pinches the left thumb and index finger to commit the command. The agent then turns and walks to that point unless an anchored wall blocks the path.

Storyboard 1 (implemented): Aim the right hand, then pinch the left to send the agent

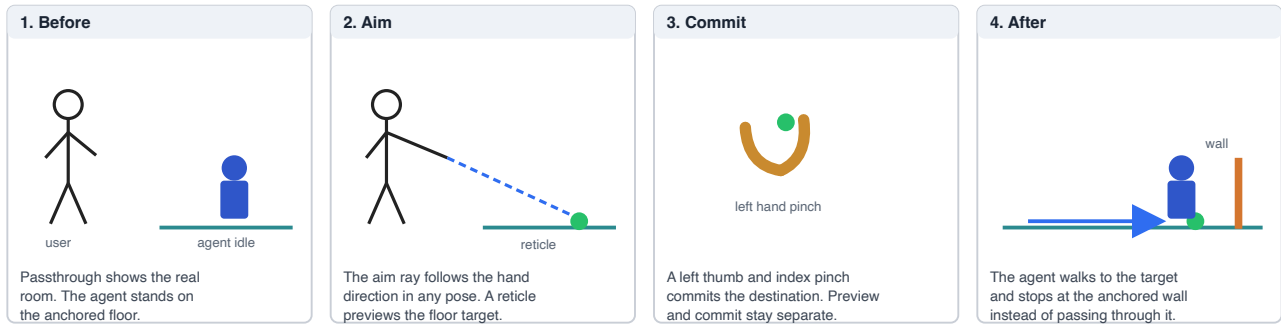


Figure 1: Storyboard 1, hand aim and pinch

Technique 2, alternative: directional swipe

The user raises an open palm to enter a command ready state, then sweeps the palm in a direction. The system reads the motion vector and moves the agent that way. This is fast and expressive, but it is easier to trigger by accident than a deliberate aim and confirm gesture.

Storyboard 2 (alternative): Open palm swipe sets the travel direction

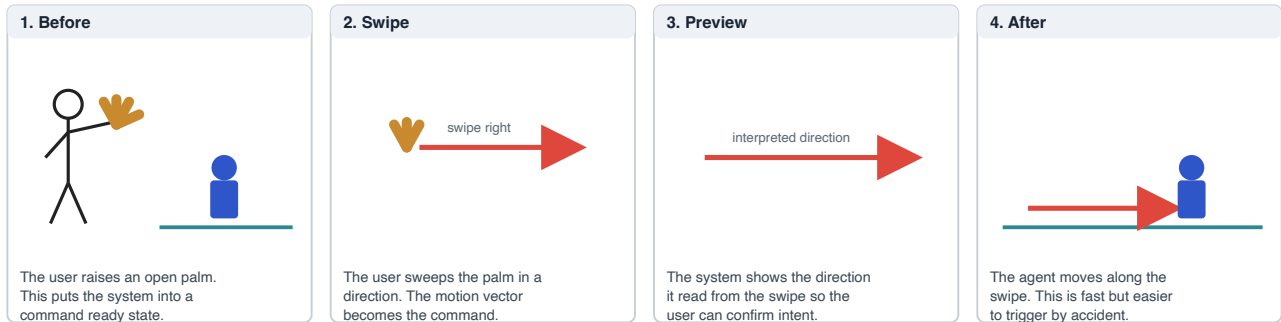


Figure 2: Storyboard 2, directional swipe

Technique 3, alternative: two hand vector

The left hand marks an origin while the right hand sets a direction. The line between the two hands becomes a travel vector, and releasing the pose commits it. This is precise and does not depend on where the user is looking, but it asks more coordination of the user than the implemented method.

Chosen interaction rationale

Technique 1 was selected because it separates preview from commit. The reticle makes the user's intention visible before anything moves, the command stays grounded in the real room because the target is a real floor point, and a deliberate pinch reduces accidental movement. It also relies only on tracked hand joints, which is the constraint set by the assignment.

Heuristic evaluation

Storyboard 3 (alternative): Two hands define an origin and a direction vector

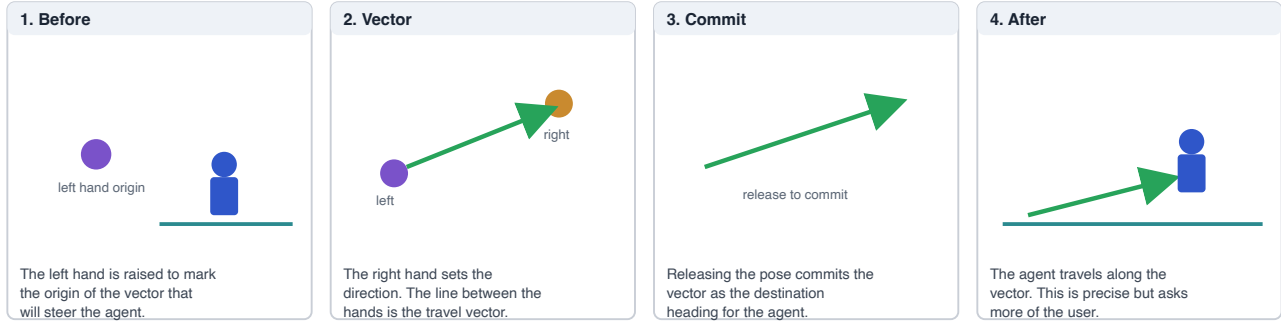


Figure 3: Storyboard 3, two hand vector

Heuristic	Violation and rationale	Severity	Recommendation
1. Visibility of system status	Mostly satisfied. The reticle, the aim ray, and the status panel show the current aim point, the captured floor and wall counts, and the agent state. A newly detected surface still needs a moment of interpretation before the user trusts it.	Low	Keep the status panel in view during scanning and label the translucent surface colors so the user reads them at a glance.
2. Match between system and the real world	Mostly satisfied. Aiming the hand at the real floor is natural, and the walls are digital copies of the user's own room, so virtual blocking matches the physical space. A partial room scan can leave a real wall without a virtual counterpart.	Medium	Prompt the user to complete a full room scan, and allow a rescan, before the agent is driven near a wall.
3. User control and freedom	Partially satisfied. A new aim and pinch at any time redirects the agent, so the user can correct a wrong command. There is no dedicated stop gesture once the agent is already walking.	Medium	Add an open palm stop gesture that halts the agent in place and clears the current destination.
4. Consistency and standards	Mostly satisfied. The right hand always aims and the left hand always confirms, and the reticle keeps the same meaning throughout. The fixed assignment of hands does not fit a left-handed user.	Low	Add a handedness toggle so either hand can take the aim role.
5. Error prevention	Mostly satisfied. The pinch confirm prevents accidental motion, and the wall colliders stop the agent before it crosses a surface. A room with no captured walls still leaves the agent free to roam past real walls.	Medium	Warn when zero walls are captured and require at least one floor and one wall before accepting a movement command.
6. Recognition rather than recall	Mostly satisfied. The visible reticle, aim ray, and status text mean the user does not need to remember system state. The one step that must be learned is the left pinch trigger.	Low	Show a short onboarding placard that names the aim hand and the pinch trigger before free interaction.
7. Flexibility and efficiency of use	Satisfied for the scope of this assignment. The gesture is quick once learned, and an editor only fallback speeds iteration without changing the Quest interaction. There is no shortcut for sending the agent through several points in sequence.	Low	Add optional waypoint queuing for expert users rather than overloading the single command.

Heuristic	Violation and rationale	Severity	Recommendation
8. Aesthetic and minimalist design	Partially satisfied. The translucent room proxies keep the real environment visible. The hand joint debug markers are visually busy once joint access is proven, and the stopped agent holds a walk pose rather than a true rest pose because the asset set has no idle clip.	Low	Hide the debug markers for the final recording and import a dedicated idle animation so the agent stands neutrally when stopped.
9. Help users recognize, diagnose, and recover from errors	Partially satisfied. When the agent is blocked the status panel reports the blocked state in plain language, but it does not point to the specific wall that stopped the agent.	Medium	Briefly highlight the blocking wall proxy when the agent stops so the cause is obvious.
10. Help and documentation	Partially satisfied. The repository README and the in-headset status panel explain how the system works, but there is no guided first-use tutorial inside the headset.	Medium	Add a concise first-use card that walks a new user through scanning, aiming, and confirming.

Implementation coverage

This section maps the graded implementation items to where each one is satisfied.

- Passthrough mode is enabled and virtual content is visible over the real world view. Satisfied.
- Spatial anchors define the key surfaces. Each captured plane creates an anchor, and the surface proxy is attached to that anchor. Satisfied.
- The physical space is modeled. A floor proxy and wall proxies are placed at the anchors as digital copies of the real surfaces. Satisfied.
- Hand joint transforms are accessed for both hands through the hand tracking subsystem. Satisfied.
- A defined gesture is detected algorithmically. The aim ray is computed from the right hand joints, and the left thumb and index pinch distance triggers the command. Satisfied.
- The destination is determined when the gesture fires. The aim ray resolves to a floor point through an AR raycast, with a physics raycast and a floor plane fallback. Satisfied.
- An animated Mixamo character is imported and driven. The Beetlejuice character walks on command. Satisfied.
- The agent responds to gestural commands and to the physical environment. It walks to the commanded point and stops at anchored walls rather than passing through them. Satisfied.